

Linux Porting and App Development is a Breeze with JTAG Debuggers

Considering the complexity of today's embedded systems, using a Linux-aware JTAG debugger vis-à-vis traditional agent based debugging can be more efficient and effective in reducing the time-to-market for Linux-based systems

T.S. JAGADISH

When it comes to embedded systems running Linux, developers are primarily concerned about two things: porting or making Linux run on the target without errors, and developing applications that can run on Linux to carry out the job the embedded system is designed for.

JTAG debuggers with Linux awareness are rising to the occasion, helping developers to debug embedded systems running Linux. This article looks at why it is difficult to use traditional methods for Linux debugging, before delving into how JTAG debuggers help sort out bugs in the different stages of Linux booting and Linux application development.

Unique challenges posed by Linux

There are several steps involved in the Linux booting process. The boot-loader first copies itself to RAM from Flash and then loads the Linux kernel. Next, the Linux kernel boots up and performs the transition from physical addresses to kernel virtual addresses.

The memory space allocation for particular processes in the physical memory may be fragmented across multiple memory regions, but these appear to be in a continuous memory address space for the process using it.

Understanding a Linux based embedded system

A Linux based embedded system can be split into three main components:

- 1. The Linux kernel.** The Linux kernel is the core of the Linux operating system (OS), having ultimate authority. It is the first component to start after the system has been initialised by the boot-loader.
- 2. Kernel modules.** Linux kernel modules are dynamically loaded and unloaded as and when needed; once a Linux kernel module is loaded it has the same level of authority as the Linux kernel. Kernel modules are mainly used for device drivers.
- 3. Application software.** Application software cannot access the Linux kernel memory or hardware directly as they run in user mode on a Linux system, with reduced privileges. So, if some application software needs to access the peripherals or memory, it needs to request the Linux kernel to provide system-level access to it.

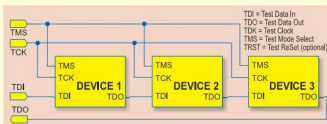


Fig. 1: Interconnection of JTAG-compliant devices in a PCB

This is done using the virtual addressing, which is maintained by memory management unit (MMU) of the Linux system.

In Linux, there is a constant switching between the kernel space and user space, and also between different processes. This makes tracking of virtual memory impossible and complicates the debugging of these processes or the Linux kernel. These issues cannot be addressed using common methods like adding 'print' statements, or agent-based debugging solutions, such as the Kernel GNU debugger (KGDB) or the

GNU debugger (GDB).

Using KGDB, a developer cannot examine the current state of the system by halting the CPU, especially in a multi-core or multi-processing environment. The breakpoints that we set in KGDB cannot halt the execution of all the processes at once as the Kernel GNU debugger (KGDB) requires a communication port, such as Ethernet (also a process), to be working continuously for the KGDB agent present in the target to communicate with the host system. This also means that to start agent based Linux kernel-debugging, we require a communication channel to be established with a working IP stack and working device driver, which may not be possible when the